

DA5300: Data Structures for Data Science

Laboratory 4 – Stacks and Queues

Objective

The objective of this laboratory is to implement and use the two fundamental linear data structures:

1. Stack
2. Queue

The laboratory will use a cricket theme to illustrate the operations performed on these data structures.

You will implement the corresponding Abstract Data Types (ADTs) using Python classes.

Part A – Stack ADT

A stack follows the **Last-In, First-Out (LIFO)** principle.

Consider a cricket team selecting players one at a time. The last player added to a stack is the first player that can be removed.

The basic Stack ADT supports the following operations.

- `S.push(e)`: Add element `e` to the top of stack `S`.
- `S.pop()`: Remove and return the top element from stack `S`. An error should occur if the stack is empty.

The following supporting operations are also required.

- `S.top()`: Return a reference to the top element of stack `S`, without removing it. An error should occur if the stack is empty.
- `S.is_empty()`: Return `True` if stack `S` does not contain any elements.
- `len(S)`: Return the number of elements currently stored in stack `S`.

Exercise 1 – Implementing a Stack

Create a class

```
class Stack:  
    ...
```

Implement the following methods.

Q1.1 Implement the constructor `__init__()`.

Q1.2 Implement

```
push(element)
```

to add an element to the top of the stack.

Q1.3 Implement

```
pop()
```

to remove and return the top element.

Q1.4 Implement

```
top()
```

to return the top element without removing it.

Q1.5 Implement

```
is_empty()
```

to determine whether the stack is empty.

Q1.6 Implement

```
__len__()
```

so that the following works:

```
S = Stack()

print(len(S))
```

Exercise 2 – Cricket Batting Order

Use the Stack class to store players who arrive at the crease.

Add the following players in the given order:

```
Virat
Rohit
Gill
Pant
Hardik
```

Answer the following questions.

Q2.1 Push all five players onto the stack.

Q2.2 Print the player at the top of the stack.

Q2.3 Pop one player.

Q2.4 Print the player now at the top.

Q2.5 Continue popping until the stack becomes empty.

Q2.6 What order are the players removed?

Q2.7 Explain why this behaviour is called **Last-In, First-Out (LIFO)**.

Exercise 3 – Stack Error Handling

Modify your implementation so that attempting to perform

```
S.pop()
```

on an empty stack produces an appropriate error.
Similarly, calling

```
S.top()
```

on an empty stack should produce an appropriate error.
Test both cases.

Part B – Queue ADT

A queue follows the **First-In, First-Out (FIFO)** principle.

Consider spectators entering a cricket stadium. The first spectator to enter the queue should be the first spectator to leave it.

The basic Queue ADT supports the following operations.

- `Q.enqueue(e)`: Add element `e` to the back of queue `Q`.
- `Q.dequeue()`: Remove and return the first element from queue `Q`. An error should occur if the queue is empty.

The Queue ADT also supports:

- `Q.first()`: Return a reference to the element at the front of queue `Q`, without removing it.
- `Q.is_empty()`: Return `True` if queue `Q` does not contain any elements.
- `len(Q)`: Return the number of elements currently stored in queue `Q`.

Exercise 4 – Implementing a Queue

Create a class

```
class Queue:  
    ...
```

Implement the following methods.

Q4.1 Implement the constructor `__init__()`.

Q4.2 Implement

```
enqueue(element)
```

to add an element to the back of the queue.

Q4.3 Implement

```
dequeue()
```

to remove and return the first element.

Q4.4 Implement

```
first()
```

to return the first element without removing it.

Q4.5 Implement

```
is_empty()
```

to determine whether the queue is empty.

Q4.6 Implement

```
__len__()
```

so that

```
Q = Queue()  
  
print(len(Q))
```

works correctly.

Exercise 5 – Cricket Stadium Queue

Consider spectators arriving at a cricket stadium.

They arrive in the following order:

```
Arun  
Bala  
Chandru  
Deepak  
Elangovan
```

Use the Queue class to model this situation.

Q5.1 Enqueue all five spectators.

Q5.2 Display the first spectator.

Q5.3 Dequeue one spectator.

Q5.4 Display the first spectator again.

Q5.5 Continue dequeuing until the queue becomes empty.

Q5.6 What order are the spectators removed?

Q5.7 Explain why this behaviour is called **First-In, First-Out (FIFO)**.

Exercise 6 – Queue Error Handling

Modify your Queue implementation so that attempting to execute

```
Q.dequeue()
```

when the queue is empty produces an appropriate error.

Similarly,

```
Q.first()
```

should produce an appropriate error when the queue is empty.

Test both cases.

Part C – Stack versus Queue

Consider the following sequence of players:

Virat
Rohit
Gill
Pant
Bumrah

Insert the players into both a Stack and a Queue in the same order.

Q7.1 Remove all players from the Stack.

Q7.2 Remove all players from the Queue.

Q7.3 Record the order in which the players are removed.

Q7.4 Explain the difference between the two data structures.

Q7.5 Which data structure would be appropriate for modelling a queue of spectators entering a stadium?

Q7.6 Which data structure would be appropriate for modelling the “undo” operation in a software application?

Part D – Implementation Challenge

Implement both **Stack** and **Queue** without using a ready-made Stack or Queue class.

You may use a Python list as the underlying storage.

Your implementation should support:

Operation	Stack	Queue
Insert	<code>push()</code>	<code>enqueue()</code>
Remove	<code>pop()</code>	<code>dequeue()</code>
Access	<code>top()</code>	<code>first()</code>
Empty check	<code>is_empty()</code>	<code>is_empty()</code>
Size	<code>len()</code>	<code>len()</code>

For each operation, determine its time complexity.

Operation	Stack	Queue
Insert		
Remove		
Access		
Empty check		
Size		